Лабораторна робота №12-13. TypeScript на практиці й AI-assisted міграція: типізація API-відповідей, union types, narrowing і type guards, any, type assertions, strict mode і runtime-валідація

Термін здачі: 19.10.2026

ЛР-12 і ЛР-13 об'єднано в одну роботу — ЛР-12-13: обидві працюють з одним застосунком.

1. Мета роботи

Навчитися типізувати фронтенд-застосунок у TypeScript: описувати відповіді API типами за документацією, типізувати функції, звужувати типи й перевіряти дані з мережі під час виконання. Навчитися переводити JavaScript-код на TypeScript з AI-агентом і перевіряти якість його результату: `any`, твердження типу, режим strict, перевірка відповідей API.

Очікуваний результат: застосунок ShowFinder повністю на TypeScript працює зі справжнім TVmaze API; `npm run check` показує «Типи: без помилок» і всі тести пройдено; типи, запити й стани ви написали власноруч, а міграцію решти модулів, зроблену AI-агентом, перевірили й виправили.

2. Цілі та результати навчання

Після виконання лабораторної роботи студент повинен:

- Описувати відповіді API інтерфейсами TypeScript за документацією й справжніми відповідями, з `null` там, де він буває.
- Типізувати функції й класи; розрізняти `unknown` і `any`; звужувати типи (`typeof`, `instanceof`, `in`, перевірка поля-дискримінанта) і писати type guards.
- Моделювати стани інтерфейсу discriminated union і обробляти їх вичерпно, з `never`.

- Читати помилки компілятора й пояснювати, як TypeScript виводить типи (type inference).
- Пояснювати, що робить `tsc`, що вмикає `strict` і чому типи зникають після компіляції.
- Пояснювати, що перевіряє, а чого не перевіряє твердження типу (`as`, `!`, `querySelector<T>()`), і замінювати його звуженням.
- Перевіряти дані з мережі під час виконання й пояснювати, чому для цього замало типу.
- Переводити JavaScript-код на TypeScript поступово (`allowJs`) з AI-агентом і перевіряти міграцію компілятором, тестами й у браузері: що агент виправив, а що лише приховав.

3. Завдання лабораторної роботи

1. Опрацювати дослідницькі завдання (§4): E1 — у браузері, E2–E6 — у проєкті, коли дійдете до відповідного завдання.
2. Завантажити starter-проєкт, зафіксувати його в Git без змін і запустити `npm run check` (§5.1).
3. Виконати завдання ТЗ, §4, по черзі: Т1–Т3 власноруч, Т4 — міграція з AI-агентом (§5.2).
4. Пройти чек-лист у браузері (ТЗ, §6) і оформити звіт (§6). Потім — захист (§7).

4. Дослідницькі завдання

Довідка: Everyday Types (зокрема Type Assertions і Non-null Assertion Operator), Narrowing, Object Types, Migrating from JavaScript — TypeScript Handbook; strict — TSConfig Reference; документація TVmaze API.

Протокол експерименту:

1. Прочитайте опис і код і запишіть, що очікуєте, — це «Мій прогноз».
2. Виконайте експеримент.
3. Якщо прогноз не збігся з результатом, поясніть, де саме він був хибним.

Результати зведіть у таблицю — вона входить у звіт:

#	Що досліджено	Мій прогноз	Фактичний результат	Пояснення (якщо не збіглося)
E1	відповідь API пошуку	…	…	…

4.1. Відповідь API — у браузері

E1. Відповідь API (перед T1). Відкрийте в новій вкладці Chrome чи Edge `https://api.tvmaze.com/search/shows?q=girls`, потім ту саму адресу з `?q=kitchen`. Браузер показує JSON; позначте **Pretty-print**, щоб його було зручно читати.

Проаналізуйте: що на верхньому рівні відповіді — об'єкт чи масив? З яких полів складається кожен елемент і де в ньому назва серіалу? У якого серіалу з `kitchen` немає постера чи дати прем'єри — і як це виглядає в JSON: поля немає чи воно `null`? Що буває `null`: сам `rating` чи `rating.average`? Що з цього має описати тип `Show` (ТЗ, T1.2)?

4.2. TypeScript — у проєкті

E2. Режим strict (після T1). Створіть тимчасовий файл `src/strict.ts`:

```
import type { Show } from './types.ts';

export function posterUrl(show: Show) {
  return show.image.medium;
}

export function shout(text) {
  return text.toUpperCase();
}
```

Запустіть `npm run typecheck`. Потім у `tsconfig.json` тимчасово замініть `"strict": true` на `"strict": false` і запустіть ще раз.

Проаналізуйте: які помилки в `src/strict.ts` покаже компілятор у кожному випадку? Що зробив би `posterUrl` для серіалу Harlem's Kitchen (id 45962) під час виконання? Без `strict` з'являється нова помилка в `check/types.check.ts`: яке правило T1 без `strict` перевірити неможливо і чому? Поверніть `"strict": true` і видаліть `src/strict.ts`.

E3. unknown, any і звуження (після T2). Наведіть курсор у редакторі на `response.json()` у `getJson`, потім у `searchShows` на дані, які повернула `getJson`: одразу після `await` і там, де ви їх уже перевірили guard-ом. Тимчасово допишіть одразу після `await` рядок `console.log(data.length);` (з вашою назвою змінної), подивіться на помилку й видаліть рядок.

Проаналізуйте: який тип має кожен із трьох виразів? Чому `getJson` оголошено як `Promise<unknown>`, а не `Promise<any>`? Що змінилося б у `searchShows`, якби вона повертала `any`?

E4. Що лишається після компіляції (після T2). Поки працює `npm run watch`, відкрийте `dist/types.js` і `dist/api.js`.

Проаналізуйте: що лишилося від `interface Show`, від `value is Show` і від `readonly status: number`? Чому `isShow` не можна написати як `value instanceof Show`? Навіщо перевіряти форму відповіді під час виконання (Т3, Т2.7), якщо типи вже є?

E5. Твердження типу проти перевірки (після Т2). Створіть тимчасовий файл `src/assert.ts`:

```
import type { Show } from './types.ts';
import { isShow } from './api.ts';

const data: unknown = JSON.parse('{ "id": 1, "name": "Pilot" }');

if (isShow(data)) console.log('guard:', data.genres.join(', '));
else console.log('guard: не серіал');

const show = data as Show;
console.log('as:', show.genres.join(', '));
```

Запустіть `npm run typecheck`, потім виконайте файл: `node src/assert.ts`. Далі допишіть у кінець рядок `const id = 'abc' as number;` і знову запустіть `npm run typecheck`. Замініть у ньому `as number` на `as unknown as number` і запустіть ще раз.

Проаналізуйте: чи знайшов компілятор помилку в рядку з `as Show`? Що виведе кожен `console.log` і де програма впаде? Чому TypeScript відмовляє `'abc' as number`, але приймає `'abc' as unknown as number`? Що насправді перевіряє `as`, а що — `isShow`? Видаліть `src/assert.ts`.

E6. Типи DOM (перед Т4). Створіть тимчасовий файл `src/dom-types.ts`:

```
const byTag = document.querySelector('form');
const byId = document.querySelector('#search');
const claimed = document.querySelector<HTMLInputElement>('#search');
const asserted = document.querySelector('#search') as HTMLInputElement;

document.addEventListener('click', (event) => {
  const target = event.target;
});
```

Наведіть курсор у редакторі на кожну змінну й на `target`. Потім на сторінці застосунку в Console виконайте `document.querySelector('#search') instanceof HTMLInputElement` і `document.querySelector('#search') instanceof HTMLFormElement`.

Проаналізуйте: який тип має кожна змінна? Звідки компілятор знає тип `byTag`, але не `byId`? `claimed` і `asserted` обіцяють поле вводу, а `#search` — це форма: хто це перевірив? Чому `event.target` — не елемент, по якому клікнули, а `EventTarget | null`? Як отримати `HTMLFormElement`, нічого не стверджуючи? Видаліть `src/dom-types.ts`.

5. Практична частина: як працювати

Starter-проєкт **ShowFinder** — пошук серіалів у TVmaze, написаний на JavaScript. Розмітка, стилі й модулі `app`, `view`, `dom` і `main` уже є — на JavaScript. Порожні три файли TypeScript: `src/types.ts`, `src/api.ts` і `src/state.ts`.

Усі завдання практичної частини — у ТЗ, §4: Т1–Т4, кожне з пронумерованими правилами, а в Т4 — ще й запит для AI-агента. Тримайте ТЗ відкритим під час роботи. Цей розділ — про те, як працювати: підготовка, правила роботи з AI-агентом і коміти.

Як перевіряти. `npm run check` спершу перевіряє типи (`tsc`), потім запускає тести (`node --test`) і закінчує підсумком «Типи: … · Тести: пройдено N з M». Назва кожного тесту й коментар біля кожної перевірки типів у `check/types.check.ts` починаються з номера правила ТЗ. Тести не ходять у мережу: вони підміняють `fetch` і відповідають записаними відповідями TVmaze, тож не залежать ні від Wi-Fi, ні від ліміту запитів. Поки модулі Т4 лишаються на JavaScript, тести Т4 виконують саме їх: частина цих тестів падає ще до міграції, бо в старому коді є дефекти. Справжній TVmaze і браузер перевіряє чек-лист ТЗ, §6.

5.1. Підготовка

1. Перевірте `node -v`: потрібен Node.js 22.18 або новіший. На старішому тести пишуть `Unknown file extension ".ts"`.
2. Завантажте showfinder-ts-starter.zip і розпакуйте в **окремий репозиторій** або в теку `lab-12/` спільного репозиторію з лабораторними. Зробіть перший коміт **без змін**: `Starter`.
3. Виконайте `npm install`, потім `npm run watch` — і лишіть його працювати в окремому терміналі.
4. Відкрийте `index.html` через Live Preview у Chrome або Edge: є форма пошуку, а в Console — `Uncaught Error: Not implemented`. Так і має бути: `app.js` викликає функції з `src/state.ts` і `src/api.ts`, яких ще немає. Сторінка запрацює після Т3.
5. Запустіть `npm run check`: на початку — багато помилок типів і «Тести: пройдено 7 з 47». Прочитайте `README.md` starter-проєкту.
6. Власноруч змінюються лише `src/types.ts`, `src/api.ts` і `src/state.ts`, а в Т4 — модулі, які переводить агент. Решта файлів, у тому числі `check/` і `tsconfig.json`, після жодного завдання не відрізняється від starter-проєкту.

5.2. Правила роботи з AI-агентом

- **Дослідження (§4).** «Мій прогноз» пишете самі, без AI-агента й до запуску.
- **T1–T3 — власноруч.** AI-агент може пояснити документацію TVmaze, помилку компілятора, тест чи поняття TypeScript, але не пише код у `src/`. Приклад запиту в режимі без редагування (ask / plan):

  > Поясни цю помилку компілятора TypeScript. Нижче — повідомлення, рядок з check/types.check.ts разом із коментарем-правилом і мій тип. Не пиши виправлений код: назви, що в моєму типі не так, і де про це прочитати в TypeScript Handbook.

- **Перед T4.** Після T3 `npm run check` показує «Типи: без помилок · Тести: пройдено 38 з 47». Решта тестів — T4: вони падають через дефекти JavaScript-модулів і через те, що ці модулі ще не на TypeScript. Виконайте експеримент E6. Самі `.js`-файли не виправляйте: це частина T4.
- **T4 — чотири кроки.**
  1. **Міграція AI.** Перемкніть AI-агента в IDE в режим, у якому він сам змінює файли проєкту (зазвичай — Agent). Закрийте в редакторі вкладки з файлами `check/`: багато агентів додають відкриті файли до контексту. Вставте в чат **лише** запит T4 з T3, слово в слово, без власних уточнень.
  2. **Коміт міграції.** Нічого не виправляючи, закомітьте **все**, що змінив агент, — навіть `tsconfig.json` чи `check/`, якщо він їх змінив: `T4: AI migration`. Комітьте, навіть якщо застосунок не працює: цей коміт показує, що зробив агент.
  3. **Перевірка.** Запустіть `npm run check`. Для кожної помилки типів чи тесту, що не пройшов, з'ясуйте, яке правило T4 чи обмеження §5 порушено і чому. Перегляньте diff коміту міграції: кожне `as`, `!`, `any`, `@ts-…` і кожна зміна `tsconfig.json` — що вони приховують? Правило T4.7 перевірте в браузері, чек-листом T3, §6.
  4. **Виправлення власноруч.** AI може пояснити тест чи помилку, але **не пише** код виправлення. Файли, яких агент не мав змінювати, поверніть командою `git restore --source=HEAD~1 -- <файл>`. Коли `npm run check` не показує жодної помилки, а чек-лист проходить, закомітьте виправлення одним комітом: `T4: fixes`.

  Якщо міграція з першого разу пройшла всі перевірки, коміту виправлень немає, а в підсумку звіту вкажіть, як ви її перевірили.

У звіті вкажіть назву та версію агента, модель і дату роботи.

5.3. Коміти

Історія Git лабораторної:

1. `Starter` — starter-проєкт без змін;
2. `T1: types`;
3. `T2: api`;
4. `T3: state`;
5. `T4: AI migration` — міграція агента без змін;
6. `T4: fixes` — якщо міграція не пройшла перевірку.

Комітити можна й частіше, але кожне завдання закінчується комітом з такою назвою. На GitHub коміт `T4: fixes` показує рівно те, що ви змінили в міграції AI: на захисті ви пояснюєте за ним, що в ній було не так. `node_modules/` і `dist/` не комітяться, а публікувати застосунок не потрібно: викладач запускає `npm run check` у вашому репозиторії.

6. Вимоги до звіту

Тема листа:

```
[Веб] ЛР-12-13: TypeScript і AI-міграція
```

У листі вкажіть репозиторій:

```
GitHub repository: https://github.com/<username>/<repository>
```

Звіт повинен містити:

1. таблицю експериментів §4 (E1–E6);
2. знімок екрана `npm run check` з підсумком «Типи: без помилок · Тести: пройдено 47 з 47»;
3. чек-лист у браузері ТЗ (В1–В4) зі знімками екрана;
4. назву та версію AI-агента, модель, дату роботи;
5. підсумок виконаної роботи: коротко, що ви написали в Т1–Т3; що показав `npm run check` після коміту `T4: AI migration` і що було не так у міграції (`any`, `as`, `!`, `@ts-…`, змінені налаштування, пропущені дефекти); що ви виправили.

7. Усний захист

Формат здачі передбачає усну співбесіду за результатами практичної реалізації. Питання нижче — типові питання співбесіди frontend-розробника з тем цієї роботи. Відповідайте

на прикладах власного коду ShowFinder, експериментів §4 і комітів `T4: AI migration` та `T4: fixes`. Викладач може попросити показати відповідь у коді чи DevTools або змінити код наживо, без AI. Як оцінюється захист — §8.

Питання для підготовки:

Типи TypeScript

- Чим `interface` відрізняється від `type`?
- Що таке структурна типізація? Чому об'єкт з API із зайвими полями підходить під `Show`?
- Що таке висновування типів (type inference)? Коли тип варто писати явно?
- Як TypeScript працює з `null` під `strictNullChecks`? Як ви описали поля, що можуть бути `null`?
- Навіщо `readonly`?

Union types, narrowing і type guards

- Що таке union type і discriminated union? Чому `ViewState` — union, а не один об'єкт з необов'язковими полями?
- Якими способами TypeScript звужує тип? Наведіть приклади з вашого коду.
- Що таке type guard (`value is Show`)? Що саме гарантує ваш `isShow`, а чого — ні?
- Навіщо змінна типу `never` у гілці `default`?
- Чому в strict-режимі змінна в `catch (err)` має тип `unknown`? Чому `instanceof TypeError` — не надто надійна ознака збою мережі?

any, unknown і твердження типу

- Чим `unknown` відрізняється від `any`? Чому `any` називають «заразним»? Звідки в коді береться `any`, якого ніхто не писав?
- Що перевіряє компілятор у `value as Show`, а що — ні? Чому `'abc' as number` — помилка, а `'abc' as unknown as number` — ні?
- Що обіцяє `!` після виразу? Чим `element!.focus()` відрізняється від `element?.focus()`?
- Чим `document.querySelector<HTMLFormElement>('#search')` відрізняється від `document.querySelector('form')`? Чому перше — теж твердження?
- Як звузити `event.target` до елемента, по якому клікнули?
- Що робить `as const` і чому §5 його дозволяє? Що перевіряє `satisfies`?

Компілятор і strict

- Що робить `tsc`? Що лишається від типів після компіляції?
- Що вмикає `strict`? Які помилки зникли в E2 і які з них були б помилками під час виконання?

- Навіщо `import type`?
- Чим `@ts-expect-error` відрізняється від `@ts-ignore`? Чому в `src/` обидва заборонені?
- Чим перевірка типів відрізняється від тестів? Що в T1–T4 перевіряє одне, а що — інше?

Перевірка даних під час виконання

- Чому тип `Show` не гарантує, що з мережі прийшов серіал?
- Де в ShowFinder межа, за якою дані вже перевірені? Чому вона в `api.ts`, а не в `view.ts`?
- Що побачить користувач, якщо TVmaze змінить формат відповіді, — з вашою перевіркою і без неї?
- Як перевірити всю форму відповіді, а не три поля? Що дають для цього бібліотеки схем, як-от Zod?

Міграція з JavaScript з AI-агентом

- Як перевести проєкт на TypeScript поступово? Що роблять `allowJs`, `checkJs` і коментар `// @ts-check`?
- Які дефекти JavaScript-модулів ShowFinder знайшов компілятор, які — лише тести, а які агент виправив сам? Чому компілятор не помітив `fetch(...).then((response) => response.json())`?
- Що в міграції агента приховувало помилку, а не виправляло її? Як ви це помітили?
- Чому агенту не можна змінювати `tsconfig.json` і `check/`? Що змінилося б, якби він вимкнув `strict`?
- Як обмежити агента наперед: що ви записали б у файл правил для агента (`AGENTS.md`) чи в налаштування лінтера (`no-explicit-any`, `no-non-null-assertion`)?

8. Оцінювання

Робота займає два слоти програми — ЛР 12 і ЛР 13, тож за неї — **4 бали** з трьох складників і **1 додатковий бал** за вчасну здачу.

Складник	Бали	Як оцінюється
Самоперевірка	1	`npm run check` у вашому репозиторії: «Типи: без помилок» і всі тести пройдено — 1; без помилок типів і пройдено щонайменше 80 % тестів — 0,5; інакше — 0
Процес	1	коміти за §5.3 і всі пункти звіту §6 — 1; бракує одного — 0,5; більше — 0
Захист	2	питання про власний код — до 1; два питання з різних груп §7 — по 0,5 за повну відповідь

Складник	Бали	Як оцінюється
Додатковий бал	+1	лист зі звітом надіслано до **19.10.2026** включно